HW5_P23

November 5, 2026

1 2

1.1 (a)

Since the expectation of a Bernoulli(r) random variable is r, we have

$$E[A] = \begin{bmatrix} pJ_n/2 & qJ_n/2 \\ qJ_n/2 & pJ_n/2 \end{bmatrix}$$

where $J_{n/2}$ is the $(n/2) \times (n/2)$ all ones matrix. Let $u = \begin{bmatrix} \mathbf{1}_{n/2} \\ \mathbf{1}_{n/2} \end{bmatrix}$, $v = \begin{bmatrix} \mathbf{1}_{n/2} \\ -\mathbf{1}_{n/2} \end{bmatrix}$. Then, $E[A] = \frac{p+q}{2}uu^\top + \frac{p-q}{2}ww^\top$. Since u and w are linearly independent, each outer product has rank 1, so $rank(E[A]) \leq 2$.

Note that $u^\top u = w^\top w = n$ and $u^\top w = 0$. Therefore, $E[A]u = \frac{p+q}{2}u(u^\top u) + \frac{p-q}{2}w(w^\top u) = \frac{n(p+q)}{2}u$. Similarly, $E[A]w = \frac{n(p-q)}{2}u$. Thus, w is an eigenvector with eigenvalue $\lambda_2 = \frac{n(p-q)}{2}$. Since $0 < q < p$, both eigenvalues must be positive. Therefore $rank(E[A]) \geq 2$, and we can combine this from the previous upperbound forces $rank(E[A]) = 2$. Then, all remaining eigenvalues are zero. Thus, $\lambda_1(E[A]) = \frac{n(p+q)}{2}$, $\lambda_2(E[A]) = \frac{n(p-q)}{2}$, and $\lambda_3, \cdots, \lambda_n = 0$.

1.2 (b)

Let $e_1, \cdots, e_n$ denote the standard basis vectors in $\mathbb{R}^n$, and define $r_{ij} = E[A_{ij}] = \begin{cases} p, & i,j \text{ in the same community} \\ q, & i,j \text{ in different communities} \end{cases}$. For $i < j$, define $X_{ij} = (A_{ij} - r_{ij})(e_ie_j^\top + e_je_i^\top)$. For diagonal entries, define $X_{ii} = (A_{ii} - p)e_ie_i^\top$. Because A is symmetric, $A - E[A] = \sum_{1 \leq i \leq j \leq n} X_{ij}$. Matrices X_{ij} are independent because random variables $(A_{ij})_{i \leq j}$ are independent. Because each X_{ij} is symmetric, $E[X_{ij}] = 0$ because $E[A_{ij} - r_{ij}] = 0$. Thus, the conditions for the matrix Bernstein are satisfied once we bound the summand norms and variance parameter.

For $i < j$, the matrix $e_ie_j^\top + e_je_i^\top$ has eigenvalues $1, -1$ and 0 for the remaining multiplicity. Therefore, $\|X_{ij}\|_{\text{op}} = |A_{ij} - r_{ij}|$. Since $A_{ij} \in 0, 1$ and $0 < r_{ij} < 1$, $|A_{ij} - r_{ij}| \leq 1$. For $i = j$, $\|X_{ii}\|_{\text{op}} = |A_{ii} - p| \leq 1$. Then, for every $i \leq j$, $\|X_{ij}\|_{\text{op}} \leq 1$. Thus, the matrix Bernstein parameter is $b = 1$.

Now we need to calculate $v = \|\sum_{i \leq j} E[X_{ij}^2]\|_{\text{op}}$. For $i < j$, $(e_ie_j^\top + e_je_i^\top)^2 = e_ie_i^\top + e_je_j^\top$. Therefore, $X_{ij}^2 = (A_{ij} - r_{ij})^2(e_ie_i^\top + e_je_j^\top)$. Since $A_{ij} \sim$ Bernoulli(r_{ij}), $E[(A_{ij} - r_{ij})^2] = r_{ij}(1 - r_{ij})$. Consequently, $E[X_{ij}^2] = r_{ij}(1 - r_{ij})(e_ie_i^\top + e_je_j^\top)$. For $i = j$, $E[X_{ii}^2] = p(1-p)e_ie_i^\top$. Thus, $\sum_{i \leq j} E[X_{ij}^2]$ is diagonal. For each vertex, there are $n/2 - 1$ other vertices in the same community, $n/2$ in the other community, and one diagonal entry. Therefore, every diagonal entry of the

variance matrix equals $(\frac{n}{2}-1)p(1-p)+p(1-p)+\frac{n}{2}q(1-p) = \frac{n}{2}[p(1-p)+q(1-q)]$. Thus, $\sum_{i\leq j} E[X_{ij}^2] = \frac{n}{2}[p(1-p)+q(1-q)]I_n$, which the operator norm of is $v = \frac{n}{2}[p(1-p)+q(1-q)]$. Because $p(1-p) \leq p$ and $q(1-q) \leq q$, $v \leq \frac{n}{2}(p+q) = d$, and thus $v \leq d$.

Now, we finally apply the matrix bernstein, for independent mean-zero symmetric matrices with $\|X_{ij}\|_{\text{op}} \leq b$, $P(\|\sum_{i\leq j} X_{ij}\|_{\text{op}} \geq t) \leq 2n\exp(-\frac{t^2}{2(v+bt/3)})$. Using $b=1, v \leq d, A-E[A] = \sum_{i\leq j} X_{ij}$, we get $P(\|A-E[A]\|_{\text{op}} \geq t) \leq 2n\exp(-\frac{t^2}{2(d+t/3)})$.

Case 1: $0 \leq t \leq d$
Since $t/3 \leq d/3$, $2(d+t/3) \leq \frac{8d}{3}$, and consequently $\frac{t^2}{2(d+t/3)} \geq \frac{3t^2}{8d}$.

Case 2: $t \geq d$
Since $d \leq t$, $2(d+t/3) \leq \frac{8t}{3}$, and consequently $\frac{t^2}{2(d+t/3)} \geq \frac{3t}{8}4$.

We can now combine the two cases to get:

$$\frac{t^2}{2(d+t/3)} \geq \frac{3}{8}\min(\frac{t^2}{d}, t)$$

and thus,

$$P(\|A-E[A]\|_{\text{op}} \geq t) \leq 2n\exp(-\frac{3}{8}\min(\frac{t^2}{d}, t))$$

where $c = \frac{3}{8}$.

From (a), we know that $\lambda_2(E[A]) = \frac{n(p-q)}{2}$. Let $\Delta = \frac{n(p-q}{2}$. Since $d \gg \log n$, the matrix Bernstein implies that $\|A-E[A]\|_{\text{op}} = O_P(\sqrt{d\log n})$. If we choose $t = C\sqrt{d\log n}$ for sufficiently large C, this makes the failure probability tend to 0. Therefore, a sufficient condition for $\|A-E[A]\|_{\text{op}} < \lambda_2(E[A])$ with high probability is for $\Delta \gg \sqrt{d\log n}$. Substitute in the definitions and simplify:

$$\frac{n(p-q)}{2} \gg \sqrt{\frac{n(p+q)}{2}\log n}n^2(p-q)^2 \gg n(p+q)\log np - q \gg \sqrt{\frac{(p+q)\log n}{2}}$$

Therefore, under $d \gg \log n$, this condition is sufficient for the noise operator norm to be smaller than the second eigenvalue with high probability as $n \to \infty$.

1.3 (c)

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import eigsh, LinearOperator

N = 1000
D = 50.0
TRIALS = 20
GAPS = np.array([0, .25, .5, .75, 1, 1.25, 1.5, 2, 3])

upper_i, upper_j = np.triu_indices(N, k=0)
same = ((upper_i < N//2) == (upper_j < N//2))
true_labels = np.r_[np.ones(N//2, dtype=bool), np.zeros(N//2, dtype=bool)]
```

```
ones = np.ones(N)
community = np.r_[np.ones(N//2), -np.ones(N//2)]

def make_adjacency(rng, p, q):
    rates = np.where(same, p, q)
    chosen = rng.random(len(upper_i)) < rates
    i, j = upper_i[chosen], upper_j[chosen]
    off = i != j
    rows = np.concatenate([i, j[off]])
    cols = np.concatenate([j, i[off]])
    return coo_matrix((np.ones(len(rows), dtype=float), (rows, cols)),
                      shape=(N, N)).tocsr()

def experiment_one(rng, gap_ratio):
    delta = gap_ratio * np.sqrt(D)
    p = (D + delta) / N
    q = (D - delta) / N
    A = make_adjacency(rng, p, q)

    # E[A] = (p+q)/2 * 11^T + (p-q)/2 * zz^T (including diagonal).
    def centered_matvec(x):
        return A @ x - ((p+q)/2 * ones.dot(x) * ones
                        + (p-q)/2 * community.dot(x) * community)
    centered = LinearOperator((N, N), matvec=centered_matvec,
                              rmatvec=centered_matvec, dtype=float)

    # largest absolute eigenvalue of symmetric centered matrix = operator norm.
    max_abs = float(np.abs(eigsh(centered, k=1, which='LM',
                                 return_eigenvectors=False, tol=1e-5)[0]))
    # largest three algebraic eigenvalues, with eigenvectors for lambda_2.
    vals, vecs = eigsh(A, k=3, which='LA', tol=1e-5)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:, order]
    predicted = vecs[:, 1] >= 0
    err = np.mean(predicted != true_labels)
    return dict(gap_ratio=gap_ratio, delta=delta, p=p, q=q,
                op_norm=max_abs, lambda2=float(vals[1]),
                lambda3=float(vals[2]), misclassification=min(err, 1-err))

def bernstein_level(p, q):
    v = N / 2 * (p*(1-p) + q*(1-q))
    return np.sqrt(2*v*np.log(4*N)) + (2/3)*np.log(4*N)

rng = np.random.default_rng(0)
records = []
for gap in GAPS:
    for trial in range(TRIALS):
```

```
        row = experiment_one(rng, gap)
        row['trial'] = trial
        row['tB'] = bernstein_level(row['p'], row['q'])
        records.append(row)
    print(f'Completed gap/sqrt(d)={gap:g}, {TRIALS} trials', flush=True)
data = pd.DataFrame(records)

rows = []
for ratio, group in data.groupby('gap_ratio', sort=True):
    rows.append(dict(gap_ratio=ratio, delta=group.delta.iloc[0],
                     op_median=group.op_norm.median(),
                     lambda2_median=group.lambda2.median(),
                     lambda3_median=group.lambda3.median(),
                     misclass_mean=group.misclassification.mean(),
                     misclass_median=group.misclassification.median(),
                     tB=group.tB.iloc[0]))
summary = pd.DataFrame(rows)
print('\nSummary:')
print(summary.round(4).to_string(index=False), flush=True)
```

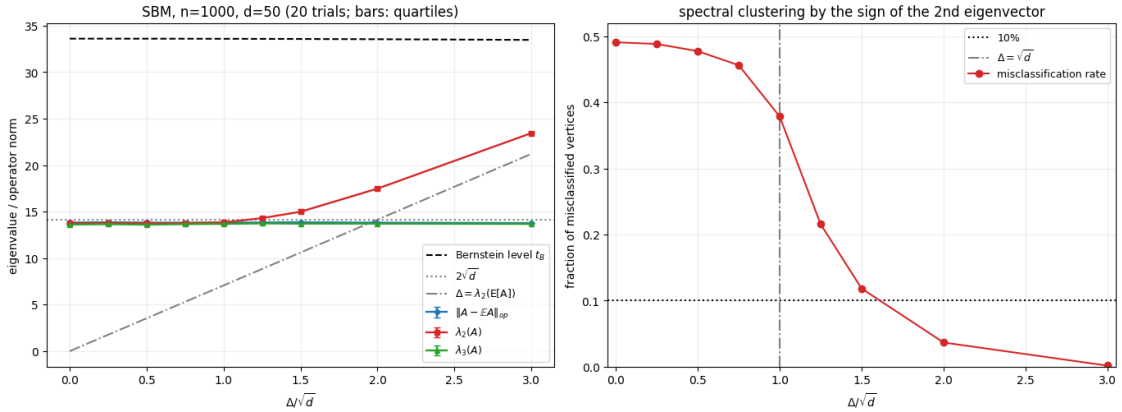
```
Completed gap/sqrt(d)=0, 20 trials
Completed gap/sqrt(d)=0.25, 20 trials
Completed gap/sqrt(d)=0.5, 20 trials
Completed gap/sqrt(d)=0.75, 20 trials
Completed gap/sqrt(d)=1, 20 trials
Completed gap/sqrt(d)=1.25, 20 trials
Completed gap/sqrt(d)=1.5, 20 trials
Completed gap/sqrt(d)=2, 20 trials
Completed gap/sqrt(d)=3, 20 trials

Summary:
 gap_ratio   delta  op_median  lambda2_median  lambda3_median  misclass_mean
misclass_median      tB
      0.00  0.0000    13.8234         13.7417         13.6114         0.4908
0.4930 33.5995
      0.25  1.7678    13.8490         13.7836         13.6559         0.4883
0.4900 33.5986
      0.50  3.5355    13.8218         13.7480         13.5995         0.4774
0.4810 33.5958
      0.75  5.3033    13.8049         13.7783         13.6607         0.4561
0.4630 33.5912
      1.00  7.0711    13.8069         13.8432         13.6830         0.3786
0.3660 33.5848
      1.25  8.8388    13.8332         14.3013         13.7322         0.2156
0.2300 33.5764
      1.50 10.6066    13.8516         14.9913         13.7118         0.1183
0.1205 33.5663
      2.00 14.1421    13.8122         17.4759         13.7099         0.0367
```

```
0.0350 33.5404
      3.00 21.2132     13.7649          23.4466          13.6781          0.0018
0.0015 33.4663
```

[2]:
```
fig, (ax, ay) = plt.subplots(1, 2, figsize=(13.6, 5.0),
                             constrained_layout=True)
x = summary.gap_ratio.to_numpy()
labels = [r'$\|A-\mathbb{E}A\|_{op}$', r'$\lambda_2(A)$', r'$\lambda_3(A)$']
markers = ['o', 's', '^']
colors = ['tab:blue', 'tab:red', 'tab:green']
for key, label, marker, color in zip(['op_norm', 'lambda2', 'lambda3'],
                             labels, markers, colors):
    grouped = data.groupby('gap_ratio')[key]
    med = grouped.median().to_numpy()
    lower = grouped.quantile(.25).to_numpy()
    upper = grouped.quantile(.75).to_numpy()
    ax.errorbar(x, med, yerr=[med-lower, upper-med],
                marker=marker, color=color, label=label, capsize=3,
                linewidth=1.7, markersize=4)
ax.plot(x, summary.tB, '--', color='black', label=r'Bernstein level $t_B$')
ax.axhline(2*np.sqrt(D), linestyle='dotted', color='gray',
           label=r'$2\sqrt{d}$')
ax.plot(x, x*np.sqrt(D), linestyle='dashdot', color='gray', label=r'$\Delta =␣
 ↪\lambda_2$(E[A])')
ax.set(xlabel=r'$\Delta/\sqrt{d}$', ylabel='eigenvalue / operator norm',
       title=f'SBM, n={N}, d={D:g} ({TRIALS} trials; bars: quartiles)')
ax.legend(fontsize=9, loc='lower right')
ax.grid(alpha=.22)
ay.axhline(.1, linestyle='dotted', color='black', label='10%')
ay.axvline(1, linestyle='dashdot', color='gray', label=r'$\Delta=\sqrt{d}$')
ay.plot(x, summary.misclass_mean, marker='o', color='tab:red',
        label='misclassification rate')
ay.set(xlabel=r'$\Delta/\sqrt{d}$', ylabel='fraction of misclassified vertices',
       title='spectral clustering by the sign of the 2nd eigenvector',
       ylim=(0, .52), xlim=(-.05, 3.05))
ay.legend(fontsize=9)
ay.grid(alpha=.22)

plt.show()
```



i. $2\sqrt{d} = 2\sqrt{50} \approx 14.14$. Across all gaps, the median noise operator norm is on average about 13.8, while t_B is on average about 33.6. Thus, the observed operator norm is much closer to $2\sqrt{d}$ than the Bernstein bound. This shows how the Bernstein bound provides a valid but reasonable upper bound for the experiment.

ii. The second eigenvalue begins visibly separating at point $\Delta/\sqrt{d} = 1.25$. The average misclassification rate first falls below 10% at point $\Delta/\sqrt{d} = 2.0$. Condition $\Delta > t_B$ requires $\Delta/\sqrt{d} > 33.6/\sqrt{50} \approx 4.75$. Condition $\Delta > \|A - E[A]\|_{\text{op}}$ requires $\Delta/\sqrt{d} > 13.8/\sqrt{d} \approx 1.95$. Condition $\Delta > \sqrt{d}$ requires $\Delta/\sqrt{d} > 1$. Thus, eigenvalue separation is closest to the 1 threshold, but misclassification rate falling under 10% is closest to the empirical noise threshold of 1.95. We can conclude that the condition $\Delta > \|A - E[A]\|_{\text{op}}$ is sufficient but not necessary for spectral clsuetring to work.

2 3

2.1 (a)

We know each X_i is uniformly distributed among the 2d vectors $\pm\sqrt{d}e_j$. Therefore, $X_i X_i^\top = de_j e_j^\top$ whenever $X_i \pm \sqrt{d}e_j$. Let N_j be the number of samples that fall on coordinate axis j. Since each axis is selected with probability $1/d$, $(N_1, \cdots, N_d) \sim \text{Multinomial}(n; \frac{1}{d}, \cdots, \frac{1}{d})$. Thus, $\hat{\Sigma} = \frac{1}{n}\sum_{i=1}^n X_i X_i^\top = \frac{d}{n}\text{diag}(N_1, \cdots, N_d) = \text{diag}(N_1, \cdots, N_d)$ in the case where $n = d$. Because $\hat{\Sigma}$ is diagonal with nonnegative entries, $\|\hat{\Sigma}\|_{\text{op}} = \max_{1 \le j \le n} N_j$, which suffices to prove that $P[\|\hat{\Sigma}\|_{\text{op}} \ge \frac{c \log n}{\log \log n}] \to 1$, since this is equivalent to the maximum occupancy when n balls are independently placed into n bins uniformly at random.

Now we choose $k = \lfloor \frac{1}{2}\frac{\log n}{\log \log n} \rfloor$, where at sufficiently large $n, k \ge 2$ and $k < n$. For each j, $N_j \sim \text{Bin}(n, \frac{1}{n})$. Then, we can use the provided lower bound on the binomial tail and set $p = \frac{1}{n}$ and $q = \frac{k}{n}$, knowing that since $k \ge 2$, we get $q > p$, and $nq = k$ is an integer. Thus, $P(N_j \ge k) \ge \frac{\exp(-nD(\frac{k}{n}\|\frac{1}{n}))}{\sqrt{8k(1-k/n)}}$. We can expand $nD(\frac{k}{n}\|\frac{1}{n}) = k\log k + (n-k)\log(\frac{n-k}{n-1})$, and since $k \ge 1$, $\frac{n-k}{n-1} \le 1$, so the log in the second term is negative. Therefore, $nD(\frac{k}{n}\|\frac{1}{n}) \le k \log k$, and we can plug this back in $P(N_j \ge k) \ge \frac{e^{-k\log k}}{\sqrt{8k}}$. By our choice of k, for sufficiently large n, $k \le \frac{1}{2}\frac{\log n}{2\log\log n}$, $\log k \le \log\log n$, so $k \log k \le \frac{1}{2}\log n$, which implies that $e^{-k\log k} \ge e^{-(\log n)/2} = n^{-1/2}$, and finally $P(N_j \ge k) \ge \frac{1}{\sqrt{8kn}}$.

Now, define an indicator random variable $I_j = \mathbf{1}_{N_j \geq k}$ and let $Z = \sum_{j=1}^{n} I_j$. Thus, Z counts the number of bins containing at least k samples, and $Z > 0 = \max_{1 \leq j \leq n} N_j \geq k$. By linearity of expectation and the buond we just established, $E[Z] = \sum_{j=1}^{n} P(N_j \geq k) = nP(N_1 \geq k) \geq \sqrt{\frac{n}{8k}}$. Since $k = O(\log n)$, we know $E[Z] \to \infty$. For $i \neq j$, conditional on $N_i = a$, $N_j | N_i = a \sim \text{Bin}(n - a, \frac{1}{n-1})$. Define $g(a) = P(N_j \geq k | N_i = a)$. As a increases, the number of remaining samples $n - a$ decreases. Therefore, $g(a)$ is non increasing, but $\mathbf{1}_{a \geq k}$ is non decreasing in a. A nondecreasing function and a nonincreasing function of the same random variable have nonpositive covariance. Thus, $Cov(I_i, I_j) \leq 0$. Since I_j is an indicator random variable, $I_j^2 = I_j$, and hence $Var(I_j) = E[I_j] - E[I_j]^2 \leq E[I_j]$. Using this, $Var(Z) = \sum_{j=1}^{n} Var(I_j) + 2 \sum_{i<j} Cov(I_i, I_j) \leq \sum_{j=1}^{n} Var(I_j) \leq \sum_{j=1}^{n} E[I_j] = E[Z]$. By Chebyshev's inequality, $P(Z = 0) \leq P(|Z - E[Z]| \geq E[Z]) \leq \frac{Var(Z)}{E[Z]^2} \leq \frac{1}{E[Z]} \leq \sqrt{\frac{8k}{n}} \to 0$. Therefore, $P(\max_{1 \leq j \leq n} N_j \geq k) = P(Z > 0) \to 1$.

Since $\lfloor x \rfloor \geq x - 1$ and $\frac{\log n}{\log \log n} \to \infty$, $k \geq \frac{1}{n} \frac{\log n}{\log \log n} - 1 \geq \frac{1}{4} \frac{\log n}{\log \log n}$. Then, by the two previous steps, $P(\|\hat{\Sigma}\|_{\text{op}} \geq \frac{1}{4} \frac{\log n}{\log \log n}) \geq P(\max_{1 \leq j \leq n} N_j \geq k) \to 1$ as $n \to \infty$ with $c = \frac{1}{4}$.

2.2 (b)

[ ]:
```
from joblib import Parallel, delayed
from threadpoolctl import threadpool_limits

from scipy.linalg import eigh

def trial(regime, d, distribution, seed):
    with threadpool_limits(limits=1):
        rng = np.random.default_rng(seed)
        n = d if regime == 'n=d' else 50 * d
        if distribution == 'Sparse':
            counts = rng.multinomial(n, np.full(d, 1.0 / d))
            error = np.max(np.abs((d / n) * counts - 1))
        else:
            # Rows are independent, uniform on the sphere of radius sqrt(d).
            X = rng.standard_normal((n, d), dtype=np.float64)
            X *= (np.sqrt(d) / np.linalg.norm(X, axis=1))[:, None]
            if d <= 256:
                covariance = (X.T @ X) / n
                extremes = eigh(covariance, eigvals_only=True,␣
 ↪subset_by_index=(0, d-1))
                lo, hi = extremes[0], extremes[-1]
            else:
                # Compute extreme eigenvalues without materializing d x d␣
 ↪matrix.
                def mv(v):
                    return (X.T @ (X @ v)) / n
                op = LinearOperator((d, d), matvec=mv, dtype=np.float64)
                hi = eigsh(op, k=1, which='LA', return_eigenvectors=False,
                           tol=1e-5, maxiter=2000)[0]
```

```
                if regime == 'n=d' and hi >= 2.0:
                    # Exact shortcut: all covariance eigenvalues are >= 0,
                    # so 1-lambda_min <= 1 <= lambda_max-1.
                    lo = 0.0
                else:
                    lo = eigsh(op, k=1, which='SA', return_eigenvectors=False,
                               tol=1e-5, maxiter=3000)[0]
            error = max(abs(hi - 1), abs(lo - 1))
        return {'regime': regime, 'd': d, 'n': n,
                'distribution': distribution, 'error': float(error)}

def bernstein(d, n):
    b = (d - 1) / n
    v = (d - 1) / n
    return np.sqrt(2 * v * np.log(4 * d)) + (2 * b / 3) * np.log(4 * d)

dimensions = {
    'n=d': [16, 32, 64, 128, 256, 512, 1024, 2048, 4096],
    'n=50d': [16, 32, 64, 128, 256, 512, 1024],
}
tasks = [(regime, d, dist, trial_idx)
            for regime, ds in dimensions.items() for d in ds
            for dist in ('Sparse', 'Spherical') for trial_idx in range(20)]
seeds = np.random.SeedSequence(0).spawn(len(tasks))
print(f'Running {len(tasks)} trials using 8 joblib workers (seed=0)',␣
 ↪flush=True)

results = Parallel(n_jobs=8, backend='threading', verbose=5)(
    delayed(trial)(regime, d, dist, seed)
    for (regime, d, dist, _), seed in zip(tasks, seeds)
)
raw = pd.DataFrame(results)
summary = (raw.groupby(['regime', 'd', 'n', 'distribution'], sort=False)
                .error.agg(median='median', q25=lambda s: s.quantile(.25),
                           q75=lambda s: s.quantile(.75), mean='mean')
                .reset_index())
print(summary.to_string(index=False, float_format=lambda x: f'{x:.4f}'))
```

```
Running 640 trials using 8 joblib workers (seed=0)
```

```
[Parallel(n_jobs=8)]: Using backend ThreadingBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done   2 tasks      | elapsed:    0.0s
[Parallel(n_jobs=8)]: Done  56 tasks      | elapsed:    0.2s
[Parallel(n_jobs=8)]: Done 146 tasks      | elapsed:    0.6s
[Parallel(n_jobs=8)]: Done 272 tasks      | elapsed:    1.4s
[Parallel(n_jobs=8)]: Done 434 tasks      | elapsed:   11.1s
```

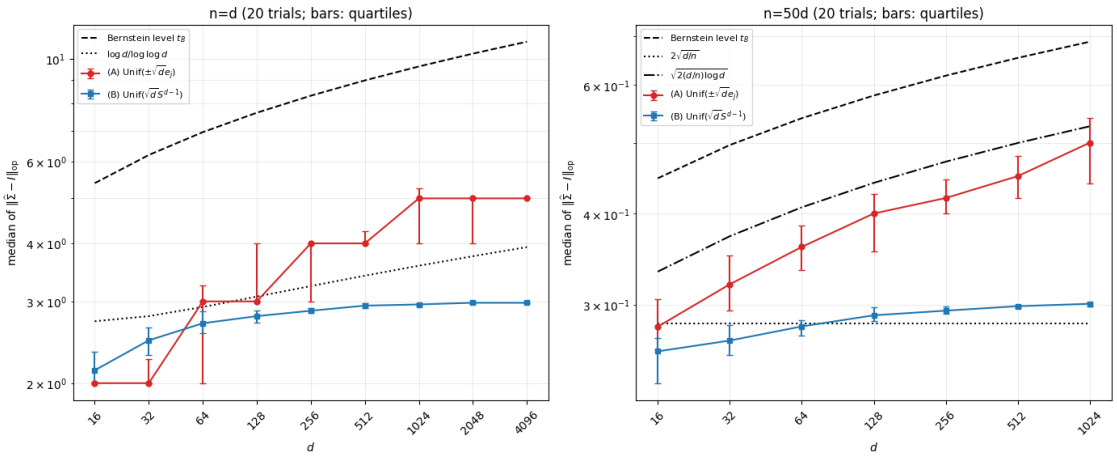
```
regime    d     n distribution  median    q25    q75   mean
   n=d   16    16       Sparse  2.0000 2.0000 2.0000 2.0000
```

```
   n=d   16    16    Spherical  2.1318 2.0124 2.3396 2.1652
   n=d   32    32       Sparse  2.0000 2.0000 2.2500 2.2000
   n=d   32    32    Spherical  2.4733 2.3012 2.6374 2.4813
   n=d   64    64       Sparse  3.0000 2.0000 3.2500 2.8000
   n=d   64    64    Spherical  2.6915 2.5640 2.8597 2.7094
   n=d  128   128       Sparse  3.0000 3.0000 4.0000 3.3500
   n=d  128   128    Spherical  2.7886 2.6994 2.8642 2.7941
   n=d  256   256       Sparse  4.0000 3.0000 4.0000 3.8000
   n=d  256   256    Spherical  2.8641 2.8277 2.9038 2.8820
   n=d  512   512       Sparse  4.0000 4.0000 4.2500 4.1500
   n=d  512   512    Spherical  2.9374 2.9049 2.9681 2.9348
   n=d 1024  1024       Sparse  5.0000 4.0000 5.2500 4.9000
   n=d 1024  1024    Spherical  2.9549 2.9385 2.9679 2.9563
   n=d 2048  2048       Sparse  5.0000 4.0000 5.0000 4.6500
   n=d 2048  2048    Spherical  2.9794 2.9634 2.9887 2.9751
   n=d 4096  4096       Sparse  5.0000 5.0000 5.0000 5.1500
   n=d 4096  4096    Spherical  2.9789 2.9723 2.9868 2.9793
 n=50d   16   800       Sparse  0.2800 0.2600 0.3050 0.2870
 n=50d   16   800    Spherical  0.2591 0.2343 0.2704 0.2539
 n=50d   32  1600       Sparse  0.3200 0.2950 0.3500 0.3290
 n=50d   32  1600    Spherical  0.2680 0.2560 0.2815 0.2702
 n=50d   64  3200       Sparse  0.3600 0.3350 0.3850 0.3790
 n=50d   64  3200    Spherical  0.2802 0.2723 0.2861 0.2807
 n=50d  128  6400       Sparse  0.4000 0.3550 0.4250 0.4060
 n=50d  128  6400    Spherical  0.2902 0.2854 0.2976 0.2919
 n=50d  256 12800       Sparse  0.4200 0.4000 0.4450 0.4300
 n=50d  256 12800    Spherical  0.2945 0.2920 0.2981 0.2957
 n=50d  512 25600       Sparse  0.4500 0.4200 0.4800 0.4610
 n=50d  512 25600    Spherical  0.2987 0.2968 0.2999 0.2990
 n=50d 1024 51200       Sparse  0.5000 0.4400 0.5400 0.5080
 n=50d 1024 51200    Spherical  0.3010 0.2987 0.3024 0.3009
```

```
[Parallel(n_jobs=8)]: Done 640 out of 640 | elapsed:  1.3min finished
```

[12]:
```
fig, ax = plt.subplots(1, 2, figsize=(13.8, 5.6), layout='constrained')
for a, regime in zip(ax, ('n=d', 'n=50d')):
    sub = summary.loc[summary.regime == regime]
    dims = np.sort(sub.d.unique())
    n_vals = dims if regime == 'n=d' else 50 * dims
    a.plot(dims, bernstein(dims, n_vals), linestyle='dashed',
           label=r'Bernstein level $t_B$', color='black')
    if regime == 'n=d':
        a.plot(dims, np.log(dims)/np.log(np.log(dims)), linestyle='dotted',
               label=r'$\log d/\log\log d$', color='black')
    else:
        a.plot(dims, 2*np.sqrt(dims/n_vals), linestyle='dotted',
               label=r'$2\sqrt{d/n}$', color='black')
        a.plot(dims, np.sqrt(2*(dims/n_vals)*np.log(dims)), linestyle='dashdot',
```

```
                label=r'$\sqrt{2(d/n)\log d}$', color='black')
    for dist, marker, label, color in (('Sparse', 'o', r'(A) Unif($\pm\sqrt{d}e_j$)',
                                        'tab:red'), ('Spherical', 's', r'(B) Unif($\sqrt{d}S^{d-1}$)', 'tab:blue')):
        dat = sub.loc[sub.distribution == dist].sort_values('d')
        middle = dat['median'].to_numpy()
        lower = middle - dat['q25'].to_numpy()
        upper = dat['q75'].to_numpy() - middle
        a.errorbar(dat.d.to_numpy(), middle, yerr=np.vstack([lower, upper]),
                   marker=marker, capsize=3, lw=1.5, markersize=5,
                   label=label, color=color)
    a.set_xscale('log', base=2)
    a.set_yscale('log')
    a.set_xticks(dims)
    a.set_xticklabels([str(int(x)) for x in dims], rotation=45)
    a.set_xlabel('$d$')
    a.set_ylabel(r'median of $\|\widehat\Sigma-I\|_{\mathrm{op}}$')
    a.set_title(f'{regime} (20 trials; bars: quartiles)')
    a.grid(True, alpha=.24, which='both')
    a.legend(fontsize=8)
plt.show()
```



i. **Regime (i)**: For the sparse distribution (A), $\|\widehat{\Sigma} - I\|_{\text{op}} = \max_{1 \le j \le d} |N_j - 1|$. As shown in part (a), the maximum occupancy of d balls placed uniformly into d bins grows on the order of $\frac{\log d}{\log \log d}$. We can see this in the plot, the median operator norm increases from 2 at $d = 16$ to 5 at $d = 4096$. Thus, distribution (A) exhibits logarithmic growth, consistent with the dimension dependent behavior of the Bernstein bound. For the spherical distribution (B), the median increases from 2.13 at $d = 16$ to approximatetly 2.98 at $d = 4096$. Therefore, the operator norm remains bounded rather than growing logarithmically.

Regime (ii): For distribution (A), $N_j \sim \text{Bin}(n, \frac{1}{d})$. The normalized occupancy fluctuations have

variance $Var(\frac{d}{n}N_j - 1) = \frac{d-1}{n} = v$. Since the maximum absolute value of d approximately Gaussian random variables with variance v is typically on the order of $\sqrt{2v\log d}$, we expect $\|\hat{\Sigma} - I\|_{\text{op}} \asymp \sqrt{v\log d}$, which we can see in the plot, increasing from 0.28 at $d = 16$ to 0.5 at $d = 1024$. For distribution (B), the operator norm follows the reference curve $\|\hat{\Sigma} - I\|_{\text{op}} \approx 2\sqrt{\frac{d}{n}} + \frac{d}{n}$. Since $n = 50d$, this comes out to roughly 0.303, which we see in the plot as the median increases from 0.2591 at $d = 16$ to 0.301 at $d = 1024$.

ii. Both distributions have identical Bernstein parameters, therefore any general bound that hold with probabilty at least 1/2 and depends only on v, b, d must apply equally to both distributions. In regime (ii), the sparse distribution (A) has operator norm of order $\sqrt{v\log d}$, which we see in the experimental results. Consequently, a bound valid for both distributions with probability at least 1/2 must satisfy $C(v, b, d) \gtrsim \sqrt{v\log d}$. However, for the spherical distribution (B), $\|\hat{\Sigma} - I\|_{\text{op}} \approx 2\sqrt{v}$ with a smaller correction term which we also see in the results. Comparing the two scales, we have $\frac{v\log d}{2\sqrt{v}} = \frac{\log d}{2}$. Thus, any bound depending only on v, b, d is too large for the spherical distribution by a factor of $\frac{\log d}{2}$. This difference is caused by the directional structure of the distributions. In distribution (A), every sample lies entirely on one coordinate axis, producing larger occupany fluctuations. In distribution (B), samples are spread uniformly across the sphere, giving much better concentration in any fixed direction. Therefore, proving the sharper bound requires additional assumptions about the distribution of X_i, such as uniform control of one-dimensional projections through sub-Gaussian concentration, rather than relying only on v and b.